Perfect Sampler for Monotone CNF Formulas

Guoliang Qiu, Yanheng Wang, Chihao Zhang

1 Introduction

It is well-known that the problem of counting/sampling satisfying assignments of a CNF formula is #P-complete. However, if we consider only the monotone CNF formulas, where all variables appear positively, the problem is tractable in a wide range of parameters. Monotone formulas are natural and expressive enough in various real-life applications, so it is of pratical interest to count or sample them.

The problem can be tackled by a simple Markov chain, namely the Glauber dynamics, that picks a random variable in each step and flips it with half probability if the clause remains satisfied. In a recent breakthrough [HSZ19], Hermon, Sly and Zhang gave an elegant rapid-mixnig proof for this Markov chain, with paramters matching the hardness bound. Their result essentially closed the topic on *approximate* sampling/counting the satisfying assignments for monotone CNF formulas.

In this paper, we present a similar construction that produces *perfect* samples of such assignments. This exact sampler is free of initialisation bias, which might find helpful in some applications. Besides, it has better performance if our goal is to drive a counting algorithm by known connections [JVV86]. Our main technical tool is the CFTP technique [PW96].

A monotone CNF ϕ can be readily transformed into a hypergraph H by taking the variables as vertices and clauses as hyperedges; and vice versa. It's clear that the number of satisfying assignments in ϕ coincides with the number of independent sets in H. To make our discussion more streamlined, we diverge our effort to counting the independent sets in a hypergraph. Our main result is the following theorem.

Theorem 1. Let H be a k-uniform hypergraph of maximum degree Δ. If $\Delta \leq 2^{k/2}/(3k^2)$, then there is a polynomial time algorithm that produces a *perfect* uniform sample of independent set of H.

2 Preliminaries

We settle the concepts and expose the key ideas in this section.

Definition of the Model

A hypergraph $H = (V, E)$ consists of a vertex set V and a hyperedge set $E \subseteq 2^V$. That is, each hyperedge (or edge for short) in E is a subset of V that "connects" possibly more

than two vertices. For any vertex $v \in V$, we write $\partial v := \{e \in E \mid v \in e\}$ to be the set of edges that connects v. The degree of vertex v is given by $|\partial v|$, and we use Δ to denote the maximum degree in the hypergraph. As a convention, n and m stands for the number of vertices and edges, respectively.

Throughout the paper, we shall only focus on k-uniform hypergraphs, in which $|e| = k$ for all $e \in E$. Our requirement that every edge connects *exactly* k vertices doesn't lead to a real loss of generality, since we could allow multisets, pad the smaller edges by replicates of vertices, and slightly modify our proof.

An independent set σ of hypergraph $H = (V, E)$ is an assignment $V \to \{0, 1\}$ satisfying $\forall e \in E, \exists v \in e : \sigma(v) = 0$. In other words, σ "picks" a vertex by assigning a 1 to it, and prevents any edge from being full. We denote by Ω the collection of all independent sets of H. In the next subsection, we will design Markov chains walking on Ω with stationary distribution π equal to uniformity.

Glauber Dynamics and Grand Coupling

We shall describe a grand coupling of Markov chains. Fix $s \leq 0$ and $\sigma \in \Omega$. We use $(X^\sigma_{s,t})_{s \leq t \leq 0}$ to denote the Markov chain starting at time s from the initial state σ. By varying the parameters s and σ, we have actually created *a family of* chains.

At time t, we draw a uniform random pair $R_t := (v_t, b_t) \in V \times \{0, 1\}$ independently to drive the chains $(X^\sigma_{s,t})$. We derive the state at $t + 1$ by the transition rule $X^\sigma_{s,t+1} = f(X^\sigma_{s,t}, R_t) :=$ "Try to reset $X^\sigma_{s,t}(v_t)$ to the proposed value b_t. If we are unlucky that the resulting state falls out of Ω, then we claim failure and return the original state; otherwise, return the new state."

The i.i.d. update sequence (R_t) serves as the working horse of the entire family of chains. All chains are driven by the same random source simultaneously and form a grand coupling. Observing individually, each chain implements the Glauber dynamics and has uniform stationary distribution over Ω.

For analysis purpose, we would also define an auxiliary family of Markov chains, $(Y_{s,t})_{t=s}^\infty$, as follows. The chain $(Y_{s,t})_{s \leq t \leq 0}$ starts at time s from state $\mathbf{1} \notin \Omega$. To run for one step, it uses the same update sequence (R_t): The state at $t + 1$ is derived by the transition rule $Y_{s,t+1} = g(Y_{s,t}, R_t) :=$ "Reset $Y_{s,t}(v_t)$ to b_t anyway and return." Note that the chains could in general *fall out of* Ω, the space of legal independent sets. Nonetheless, they are still a useful upper bound on the chains $(X^\sigma_{s,t})$, which will be clear in the following lemma.

Lemma 2. For all $r \leq s \leq t \leq 0$ and $\sigma \in \Omega$, we have $X^\sigma_{r,t} \leq Y_{r,t} \leq Y_{s,t}$.

Proof. Recall that all the chains update the same vertex v_t at time t. (i) If the update succeeds in $X^\sigma_{r,t}$, then $X^\sigma_{r,t+1}(v_t) = Y_{r,t+1}(v_t) = Y_{s,t+1}(v_t) = b_t$. (ii) If the update fails in $X^\sigma_{r,t}$, then $b_t = 1$ and $X^\sigma_{r,t+1}(v_t) = 0 \leq 1 = Y_{r,t+1}(v_t) = Y_{s,t+1}(v_t)$. Thus the partial order is preserved in either case. A simple induction proves the lemma. $\square$

Random Map and CFTP Sampler

When describing a CFTP sampler, it is convenient to abstract away the explicit update sequence by a random map. To be specific, we denote $f_t(\cdot) := f(\cdot, R_t)$, which essentially

bakes the randomness into the transition function. We further define $F_c^d := f_{d-1} \circ \cdots \circ f_c$ to be the composed random map bridging from time c to time d ($c < d \leq 0$). The key issue in designing a CFTP algorithm is to construct an efficient method for detecting the coalescence of $F_c^d(\Omega)$, i.e. telling whether $|F_c^d(\Omega)| = 1$. Provided such ability, a CFTP sampler follows immediately:

$j := 0$
repeat
  $j := j + 1$
  generate $R_{-jT}, \ldots, R_{-jT+T-1}$ independently
until we detect that $|F_{-jT}^{-jT+T}(\Omega)| = 1$;
return the unique element in $F_{-jT}^0(\Omega)$

where the *block size* T is a constant depending on H. We shall determine it later in the paper.

We have some comments on the termination condition of the algorithm. A standard CFTP sampler would rather detect the condition $|F_{-jT}^0(\Omega)| = 1$. In our version, however, we only examine a sufficient condition of it (with no detriment to the correctness, of course). Such treatment fits better in our analysis, but it is not a neccessity.

Key Idea

The key idea is to exploit the time-space property assuming $|F_c^d(\Omega)| > 1$. Roughly speaking, we could somehow reduce the coalescence detection problem to the reachability problem on a specific graph, with bounded one-side error. If the reachability problem has a negative answer, then we claim for sure that the image space has collapsed. We take care of the reduction in Section 3, and lower-bound the probability that we get such a negative answer in Section 4. Combining these two parts, we are able to detect coalescence efficiently in expectation.

3 Discrepancy Sequence and Coalescence Detection

To lighten our notation, we denote $X_t^\sigma := X_{c,t}^\sigma$ in this section. Suppose F_c^d doesn't coalesce Ω to a singleton; that is, there exist $\sigma, \tau \in \Omega : X_d^\sigma \neq X_d^\tau$. Let us look back in time and space to see what happened.

We take an arbitrary vertex $u : X_d^\sigma(u) \neq X_d^\tau(u)$. Without loss of generality, assume $X_d^\sigma(u) = 0 \neq 1 = X_d^\tau(u)$. Let $t := (c-1) \vee \sup\{t < d \mid v_t = u\}$ be the previous time *we are about to update* u. If $t \geq c$, why didn't the two chains agree at v after the update? The only possibility is that the proposed $b_t = 1$ fails in the first chain while succeeds in the other. Hence, there is some $e \in \partial v : X_t^\sigma(e \setminus \{v\}) = \mathbf{1}$ that prevents the first chain from promoting. On the other hand, e didn't cause problem in the second chain, so there exists $u' \in e \setminus \{u\} : X_t^\tau(u') = 0 \neq 1 = X_t^\sigma(u')$.

We collect the information into a 3-tuple (t, u, e) standing for "The two chains diagree at vertex u because the previous update at time t succeeds in one chain but fails in the other. The edge e accounts for the failure."

Since we have spot a vertex u' that distinguishes the two chains at time t, the above procedure can be repeated (by replacing d, u by t, u') until we hit the boundary time $c-1$. Finally, we obtain a *discrepancy sequence* $\zeta = ((t_i, u_i, e_i))_{0 \le i \le \ell}$ moving along the space-time. (The absence of e_0 is inconsequential.) The next lemma summarises the above discussion, and enumerates some properties of ζ.

Lemma 3. If $|F_c^d(\Omega)| > 1$, then there is a non-empty sequence $\zeta = ((t_i, u_i, e_i))_{0 \le i \le \ell}$ and states $\sigma, \tau \in \Omega$ satisfying (1) $c - 1 = t_0 < t_1 < \cdots < t_\ell < d =: t_{\ell+1}$; (2) For all $0 < i \le \ell$,

(2a) $R_{t_i} = (v_{t_i}, b_{t_i}) = (u_i, 1)$. In addition, the vertex u_i is *not* updated during the whole period $(t_i, t_{i+1}]$.

(2b) $u_{i-1} \in e_{i-1} \cap e_i$.

(2c) Either $X_{t_i}^\sigma(e_i \setminus \{u_i\}) = \mathbf{1}$ or $X_{t_i}^\tau(e_i \setminus \{u_i\}) = \mathbf{1}$, but not both. As a boundary case, $X_c^\sigma(u_0) \neq X_c^\tau(u_0)$.

Remark. In fact, these properties exactly characterises a discrepancy sequence, and the converse of the lemma is true. But this is not crucial for our purpose.

The lemma suggests a method for detecting the coalescence of $F_c^d(\Omega)$. Unfortunately, there's no obvious way to decide property (2c) efficiently, since we don't know in priori what σ and τ are, unless we try out $|\Omega|$ possibilities. In order to ease the decision, we relax the lemma by utilising monotonicity. We notice that properties (2a), (2c) and Lemma 2 directly imply the weaker property

(2d) $Y_{s,t_i+1}(e_i) = \mathbf{1}$ for all $c \le s \le t_i + 1$

since the Y-chain never fails when setting the value of u_i to 1. This condition is much easier to decide than (2c) since $(Y_{s,t})$ starts from state $\mathbf{1}$ by definition, eliminating the need for an exhaustive search. We will drop property (2c) from now on.

To make use of the lemma algorithmically, we will construct a directed graph $\tilde{G} = (\tilde{V}, \tilde{E})$ that displays clear correspondence to the properties. It comes handy to cut the block $[c, d]$ into time slices of length $L := kn$. Here we assume that $L \mid d - c$ to avoid clutter. We denote the number of slices as $N := (d-c)/L$, and the head of slice i as $P(i) := c + iL$.

Remark. We stress that the slicing technique is not necessary for the construction of $\tilde{G}$ to work. It is essential, however, in our later analysis to control probability, so we adopt it in our construction to simplify notation.

Now we construct the graph. We take $\tilde{V} := \{0, 1, \ldots, N\} \times E$. $((i, e), (i', e')) \in \tilde{E}$ iff $0 \le i' - i \le 1$ and $e \cap e' \neq \emptyset$. Intuitively, the first coordinate stands for the index of time slice. Each edge in $\tilde{E}$ only interacts between adjacent time slices. These requirements obviously reflect properties (1) and (2b). The next definitions incorporates properties (2a) and (2d) in a relaxed sense.

Definition 1. Let $(i, e) \in \tilde{V}$. We call (i, e) *susceptible* if some $v \in e$ was not updated throughout the period $[P(i), P(i+1))$. We call (i, e) *active* if some $v \in e$ got updated at

some time $t \in [P(i), P(i+1))$, and in addition $Y_{P(i-1),t+1}(e) = \mathbf{1}$. As a convention, we call $(0, e)$ active for all e.

Based on these, we call (i, e) bad if there is some $0 \le i' \le i$ such that (i', e) is active and $(i'+1, e), \ldots, (i, e)$ are susceptible. In words, a site is bad iff it is at the top of a (possibly empty) stack of susceptible sites which in turn mounts on an active site.

Remark. It seems peculiar that we require the Y-chain to start from the previous slice rather than from c. This turns out to be crucial that it gives us independence during time analysis in Section 4 – we may ignore the effect of remote updates.

Lemma 4. If there is a sequence ζ satisfying properties (1)(2a,b,d), then we can find a path γ in $\tilde{G}$ from $\{0\} \times E$ to $\{N\} \times E$ on which every sites are bad, and additionally, there is no "shortcuts" in $\tilde{E}$ that connects two non-adjacent sites in γ.

Proof. First we shall construct a path γ from $\{0\} \times E$ to $\{N\} \times E$ based on $\zeta = ((t_i, u_i, e_i))_{0 \le i \le \ell}$. Then we will verify that every sites on γ are indeed bad. Finally we rectify γ a bit to meet the additional requirement.

The construction of γ is almost self-evident: Chain the sites $(\lfloor t_i/L \rfloor, e_i)$ in series where i advances from 1 to ℓ, and put a head $(0, e_1)$ to the beginning. But we must repair γ a little, since $\lfloor t_{i+1}/L \rfloor$ could leap from $\lfloor t_i/L \rfloor$ by more than one. If that were the case, then we stack some dummy sites on top of $(\lfloor t_i/L \rfloor, e_i)$ to fill the gap. After the repair, γ is valid path on $\tilde{G}$ because property (2b) guarantees that it moves along an existent edge in $\tilde{E}$ at each step.

Next we check that every sites on γ are bad. The head site $(0, e_1)$ is of course bad by our convention. As for all sites $(\lfloor t_i/L \rfloor, e_i)$, properties (2a) and (2d) ensure their badness. What remain are those dummy sites. These sites are susceptible because no update occurred in their time slices. But they are directly on top of an active site, so by definition they are bad as well.

Finally, we remove the shortcuts in γ, while keeping the former arguments valid. For each $(i, e) \in \gamma$, we list all its *indirect* successors (i', e') in γ. If $((i, e), (i', e')) \in \tilde{E}$, i.e. a shortcut exists, then we remove from γ subpath in between (i, e) and (i', e'), and then glue the shortcut instead. This operation will neither disconnect γ nor turn bad sites into good ones. We repeat this procedure (conceptually) until we cannot spot shortcuts any more. This concludes the proof of the lemma. □

Combining Lemmas 3 and 4, we are able to detect the coalescence efficiently. First, construct the graph $\tilde{G}$ as prescribed, whose size $|\tilde{V}|$ is $O(m \cdot N)$; N will be chosen $O(\log n)$ in the next section. Second, mark the susceptible and active sites in $\tilde{G}$ by looking at the random source $(R_t) = ((v_t, b_t))$. For each site (i, e):

```
S := e
compute Y_{P(i-1),t} for P(i-1) ≤ t ≤ P(i+1)
for t = P(i), ..., P(i+1) - 1 do
    if v_t ∈ e then
        S := S \ {v_t}
        mark (i, e) as active if Y_{P(i-1),t+1}(e) = 1
mark (i, e) as susceptible if S ≠ ∅
```

which finishes in $O(Lk^2) = O(nk^3)$ time. Finally, solve the reachability problem from $\{0\} \times E$ to $\{N\} \times E$ with the extra restriction that the path visits bad sites only. A breadth first search suffices here: In each move, we may either walk to an adjacent active site freely, or to an adjacent susceptible site vertically. We claim $|F_c^d(\Omega)| = 1$ if no path is found at last. This completes the description of our CFTP sampler.

4 Bounding the Running Time

Let us analyse the probability that an active/susceptible site could occur, a key step towards the time bound of the algorithm.

Lemma 5. We use $\mathsf{S}(i, e)$ to denote the event that (i, e) is susceptible, and $\mathsf{A}(i, e)$ the event that (i, e) is active. Then,

$$\mathbb{P}[\mathsf{S}(i, e)] \leq \frac{k}{e^k}, \qquad \mathbb{P}[\mathsf{A}(i, e)] \leq \frac{k}{e^k} + \frac{k^2}{2^k}$$

Moreover, let $\tilde{U} \subseteq \tilde{V}$, if (i, e) and (i', e') are not adjacent in $\tilde{G}$ for all $(i', e') \in \tilde{U}$, then the events $\mathsf{S}(i, e)$, $\mathsf{A}(i, e)$ are independent of $\{\mathsf{S}(i', e'), \mathsf{A}(i', e') \mid (i', e') \in \tilde{U}\}$.

Proof. Recall that $\mathsf{S}(i, e)$ iff some $v \in e$ is not updated in time slice $[P(i), P(i + 1))$. Fix $v \in e$. The probability that v is not updated in the slice equals $(1 - 1/n)^L$. Since we take $L = kn$, this number is clearly bounded by $(1/e)^k$. Because $|e| = k$, taking union bound yields $\mathbb{P}[\mathsf{S}(i, e)] \leq k/e^k$.

To prove the second inequality, we write

$$\begin{aligned}
\mathbb{P}[\mathsf{A}(i, e)] &= \mathbb{P}[\mathsf{A}(i, e) \mid \mathsf{S}(i - 1, e)] \cdot \mathbb{P}[\mathsf{S}(i - 1, e)] \\
&\quad + \mathbb{P}\left[\mathsf{A}(i, e) \mid \overline{\mathsf{S}}(i - 1, e)\right] \cdot \mathbb{P}\left[\overline{\mathsf{S}}(i - 1, e)\right] \\
&\leq \mathbb{P}[\mathsf{S}(i - 1, e)] + \mathbb{P}\left[\mathsf{A}(i, e) \mid \overline{\mathsf{S}}(i - 1, e)\right]
\end{aligned}$$

As we have already shown, the first term is bounded by k/e^k. For the second term, notice that $\overline{\mathsf{S}}(i - 1, e)$ iff *every* $v \in e$ has been updated in the time slice $[P(i - 1), P(i))$. Under this condition and by our rule of update, at any fixed time $t \geq P(i)$, the random variables $Y_{P(i-1),t}(v)$ are in fact mutually independent Bernoulli(1/2) variables for all $v \in e$. Also recall that $\mathsf{A}(i, e)$ iff there exists $v \in e$ being updated at some $t \in [P(i), P(i+1))$ and $Y_{P(i-1),t+1}(e) = \mathbf{1}$. Thus the second term is bounded by

$$\sum_{P(i) \leq t < P(i+1)} \sum_{v \in e} \frac{1}{n} \cdot \frac{1}{2^k} = \frac{k^2}{2^k}$$

where the first two summations iterates over the possible activation time and vertex. Combining the two terms proves the inequality.

Finally, the claim of independence holds for trivial reasons: Definition 1 is rather local in both time and space. If $\tilde{U}$ and (i, e) are not adjacent, then either they are too far away in time, or they don't share common vertices in H. In both cases, we have the desired independence. $\square$

Lemma 5 provides clues that the existence of path γ is rare when k is large enough. The next theorem formalise our intuition.

Theorem 6. Assume $\Delta \leq 2^{k/2}/(3k^2)$. Let p be the probability that the path γ in Lemma 4 exists. Then $p < 1/2$ when N is of order $\log n$.

Proof. Fix a specific shortcut-free (not necessarily bad) path $\beta = ((i_1, e_1), \ldots, (i_r, e_r))$ from $\{0\} \times E$ to $\{N\} \times E$. The e_t's here should not be confused with Lemma 3. Clearly $r \geq N$. What's the probability that β is bad? This is given by

$$\begin{aligned}
\mathbb{P}[\beta \text{ is bad}] &\leq \mathbb{P}\left[\bigcap_{t=1}^{r} \mathsf{S}(i_t, e_t) \cup \mathsf{A}(i_t, e_t)\right] \\
&\leq \mathbb{P}\left[\bigcap_{t=1}^{r/2} \mathsf{S}(i_{2t}, e_{2t}) \cup \mathsf{A}(i_{2t}, e_{2t})\right] \\
&= \prod_{t=1}^{r/2} \mathbb{P}\left[\mathsf{S}(i_{2t}, e_{2t}) \cup \mathsf{A}(i_{2t}, e_{2t})\right] \\
&\leq \left(\frac{2k}{e^k} + \frac{k^2}{2^k}\right)^{r/2} < \left(\frac{2k^2}{2^k}\right)^{r/2}
\end{aligned}$$

where the second inequality drops the odd terms to gain the independence, and the last two lines follow from Lemma 5 and the fact that β is shortcut-free.

How many β are there? This number depends on the structure of $\tilde{G}$ only. An upper-bound is given by $n \cdot (2k(\Delta - 1) + 1)^r$, where n is the possible choices of starting points in slice 0, $k(\Delta - 1)$ accounts for the maximum number of edges intersecting the current edge, the factor 2 accounts for the next slice index ($+0$ or $+1$), and the plus one represents "staying at the current edge and moving to the next slice".

Multiplying the two estimates together (i.e. applying a union bound), we obtain

$$p < n \cdot \left(\frac{8k^4\Delta^2}{2^k}\right)^{r/2} \leq n \cdot \left(\frac{8k^4\Delta^2}{2^k}\right)^{N/2}$$

Under our assumption that $\Delta \leq 2^{k/2}/(3k^2)$ and taking $N \geq 2\log(2n)/\log(9/8)$, the probability p can be made smaller than $1/2$. This finishes the proof. $\square$

As a direct consequence, we can detect $|F_c^d(\Omega)| = 1$ with high probability when $d - c = N \cdot L = O(nk \log n)$. Therefore, we may simply choose the block size $T := O(nk \log n)$ in our algorithm, and the expected number of iterations is naturally bounded by a constant. Since each iteration runs in the same polynomial time (by running the breadth first search on the same graph $\tilde{G}$), the running time of the entire algorithm is bounded by a polynomial.

5 Further Discussions

Optimising Performance

The performance bottleneck of our CFTP sampler lies in the coalescence detection routine. To speed up, we first notice that we can generate the graph structure $\tilde{G}$ once and for all. Next we turn to optimise the site-marking process.

For a fixed $i \in \{0, 1, \ldots, N\}$, it is wasteful to compute the Y-chain over and over again when e is varying; once is enough. Futhermore, we can keep track of $|Y_{P(i-1),t}(e)|$, the number of 1's in $Y_{P(i-1),t}(e)$ for all e during the simulation. This incurs a Δ overhead in each step of simulation, since each vertex has influence on at most Δ edges. Now that we have access to this information, the condition $Y_{P(i-1),t}(e) = \mathbf{1}$ can be decided in constant time. Finally, we aggregate the work for all edges in the for-loop. The final effect is shown below:

$\#_t(e) := |e|$ for all t and e
for $t = P(i-1), \ldots, P(i+1) - 1$ **do**
  $\delta := b_t - Y_{P(i-1),t}(v_t)$
  foreach $e \in \partial v_t$ **do**
    $\#_{t+1}(e) := \#_t(e) + \delta$
$S[e] := e$ for all e
for $t = P(i), \ldots, P(i+1) - 1$ **do**
  foreach $e \in \partial v_t$ **do**
    $S[e] := S[e] \setminus \{v_t\}$
    mark (i, e) as active if $\#_{t+1}(e) = |e|$
foreach $e \in E$ **do**
  mark (i, e) as susceptible if $S[e] \neq \emptyset$

With the aid of appropriate data structure for set $S[e]$, we can mark all sites (e, i) in time $O(L\Delta)$ for a fixed i. Therefore, the total running time of the site-marking process is reduced to $O(N \cdot L\Delta) = O(k\Delta \cdot n \log n)$, a significant improvement over the original bound $O(k^3 m \cdot n \log n)$.

Porting to Fugacity λ

Our method also ports to the case of fugacity λ. In such scenario, we are required to generate samples according to the distribution $\pi(\sigma) := \lambda^{|\sigma|}/Z_\lambda$, where Z_λ is a normalising factor. Taking $\lambda = 1$ degenerates the model to the uniform case.

We modify the random source so that b_t follows a Bernoulli$(1 - \lambda)$ distribution. It's easy to see that π is the stationary distribution of the X-chains driven by such random source. Lemmas 3 and 4 holds as before. The running time analysis differs a little, though. We denote $\alpha := k/(1+\lambda)$, and take the slice length $L := \alpha n$. The probabilities (i, e) being susceptible/active are thus bounded by $\frac{k}{e^\alpha}$ and $\frac{k}{e^\alpha} + \frac{\alpha k}{e^\alpha}$, respectively. Therefore

$$p < n \cdot (2k(\Delta - 1) + 1)^r \left(\frac{k(2+\alpha)}{e^\alpha} \right)^{r/2} < n \cdot \left(\frac{4k^3 \Delta^2 \alpha}{e^\alpha} \right)^{r/2}$$

by the same sort of argument as before. This can be made small by enforcing $\Delta < e^{\alpha/2}\sqrt{1+\lambda}/(2k^2)$.

References

[HSZ19] Jonathan Hermon, Allan Sly, and Yumeng Zhang. Rapid mixing of hypergraph independent sets. *Random Structures & Algorithms*, 54(4):730–767, 2019.

[JVV86] Mark R Jerrum, Leslie G Valiant, and Vijay V Vazirani. Random generation of combinatorial structures from a uniform distribution. *Theoretical computer science*, 43:169–188, 1986.

[PW96] James Gary Propp and David Bruce Wilson. Exact sampling with coupled markov chains and applications to statistical mechanics. *Random Structures & Algorithms*, 9(1-2):223–252, 1996.